

CENTRO UNIVERSITARIO DE CIENCIAS EXACTAS E INGENIERIA



Centro Universitario De Ciencias Exactas e Ingenierías

Análisis de Algoritmos

Fecha: 07/10/2025

Tema: Act03 - Análisis de Clustering con TMAP en base de datos

Nombre del alumno: Rosas Arreola Ricardo Alejandro.

Sección: D01

NRC / Clave: 204835 / IL355

Código: 222967177

Carrera: Ingeniería en Computación.

División de Tecnologías para la Integración Ciber-Humana

Maestro/a: JORGE ERNESTO LOPEZ ARCE DELGADO.

Actividad #3

Tabla de contenido

Introducción	3
Objetivo.....	4
Desarrollo	5
Conclusión.....	17
Referencias.....	18

Introducción

La reducción de la dimensionalidad hace referencia hacia un método el cual ayuda a representar un conjunto de datos mientras se utiliza la menor cantidad de características posibles, siempre y cuando no perdamos las propiedades mas importantes y originales de estas. Para conseguir esto, es necesario eliminar datos que podríamos decir “basuras”, o sea que son innecesarios o irrelevantes.

Esto nos favorece para la generalización de una gran cantidad de datos, pero la precisión puede flanquear y la generalización también mientras apuntemos a cada vez más datos.

El TMAP nos ayuda a entender como se maneja el clustering de manera más visual y práctica, para una mejor interpretación y análisis de resultados, el cómo se relacionan, como son sus patrones, entre otras ventajas que nos beneficia esta herramienta.

Actividad #3

Objetivo

Debemos de aplicar una técnica de reducción de dimensionalidad, el cual se nos indico que es a partir de la herramienta de TMAP usando para este propósito una base de datos, debemos de hacer un análisis completo sobre cómo están estructurados los clusters que se generan. Y para una visualización mas clara y precisa, hacer un TMAP hacia un elemento individual para apreciar y descubrir posibles subclusters, y cuáles son las imágenes asociadas sobre estas.

En palabras mas sencillas, utilizar TMAP para la clustering de una base de datos a elección, descubrir y aprender sobre los clusters y subclusters, las imágenes asociadas y las agrupaciones que existen, de manera general y especifica (un cluster a elección).

Desarrollo

Proceso:

Realmente fue un caos para hacer funcionar el código, de manera sincera no entendía muy bien como implementarlo en código, todo lo que intentaba tronaba, entonces con ayuda de muchas librerías y guías, además de varias consultas de ia, pude procesar al menos lo requerido para este programa.

Primero, navegamos a la carpeta correspondiente en donde esta el programa/proyecto generado:

cd C:\Users\Ale\Desktop\Uni\AnálisisdeAlgoritmos\fashion-analysis

Despues, construimos la imagen con ayuda del Docker-Compose:

C:\Users\Ale\Desktop\Uni\AnálisisdeAlgoritmos\fashion-analysis>docker-compose build

```
C:\Users\Ale\Desktop\Uni\AnálisisdeAlgoritmos\fashion-analysis>docker-compose build
time="2025-10-07T21:30:17-06:00" level=warning msg="C:\Users\Ale\Desktop\Uni\AnálisisdeAlgoritmos\fashion-analysis\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] Building 4.1s (15/15) FINISHED
=> [internal] load local bake definitions          0.0s
=> => reading from stdin 640B                    0.0s
=> [internal] load build definition from Dockerfile 0.0s
=> => transferring dockerfile: 972B                0.0s
=> [internal] load metadata for docker.io/library/python:3.9-slim 0.7s
=> [auth] library/python:pull token for registry-1.docker.io 0.0s
=> [internal] load .dockerignore                  0.0s
=> => transferring context: 102B                   0.0s
=> [internal] load build context                  0.0s
=> => transferring context: 11.33MB                 0.0s
=> [1/7] FROM docker.io/library/python:3.9-slim@sha256:151b796af855298f244bc4d283bc19e19b0e638aa26c4fed2fc6809e 0.0s
=> => resolve docker.io/library/python:3.9-slim@sha256:151b796af855298f244bc4d283bc19e19b0e638aa26c4fed2fc6809e 0.0s
=> CACHED [2/7] WORKDIR /app                      0.0s
=> CACHED [3/7] RUN apt-get update || true 66 apt-get install -y --no-install-recommends build-essential 0.0s
=> CACHED [4/7] COPY requirements.txt .            0.0s
=> CACHED [4/7] RUN pip install --no-cache-dir --upgrade pip 66 pip install --no-cache-dir -r requirements.txt 0.0s
=> [6/7] COPY . .                                  0.3s
=> [7/7] RUN mkdir -p data results images          0.3s
=> exporting to image                             1.0s
=> => exporting manifest sha256:4b75902c2e9a213957d29f2b8afbc280d51e93898ca4af96b7be209a6dbb235 1.0s
=> => exporting config sha256:b58db028b2187954c22917dc6d62fd3bb3f78ebfa8d4d32d4ec4b4dd710edc0 0.0s
=> => exporting attestation manifest sha256:517125a132ef80f219a2b3601a24917b67290e7fd6561f7ff92a183d2e8ab1 0.0s
=> => exporting manifest list sha256:6ba5d204083c08632b04bca4d32f7b0d6f961197697addc3403314b99000245 0.0s
=> => naming to docker.io/library/fashion-analysis-fashion-analysis:latest 0.0s
=> => unpacking to docker.io/library/fashion-analysis-fashion-analysis:latest 0.2s
=> => resolving provenance for metadata file 0.0s
[+] Building 1/1
✓ fashion-analysis-fashion-analysis Built 0.0s
```

Ejecutamos el análisis (en otras palabras, podría decirse que ejecutamos el programa en general)

C:\Users\Ale\Desktop\Uni\AnálisisdeAlgoritmos\fashion-analysis>docker-compose up

```
[+] Running 2/2
✓ Network fashion-analysis_default Created 0.0s
✓ Container fashion-mnist-analysis Created 0.0s
Attaching to fashion-mnist-analysis
fashion-mnist-analysis | Cargando datos...
fashion-mnist-analysis | Datos cargados: 10000 muestras, 784 características
fashion-mnist-analysis | Distribución de etiquetas: (array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9]), array([1000, 1000, 1000, 1000, 1000, 1000, 1000, 1000, 1000, 1000]))
fashion-mnist-analysis | Aplicando UMAP al conjunto completo...
fashion-mnist-analysis | ✓ Gráfica guardada en: results/ e images/
fashion-mnist-analysis | =====
fashion-mnist-analysis | Analizando subclusters para categoria 5
fashion-mnist-analysis | =====
fashion-mnist-analysis | Analizando cluster 5...
fashion-mnist-analysis | /usr/local/lib/python3.9/site-packages/sklearn/cluster/_kmeans.py:1412: FutureWarning: The default value of 'n_init' will change from 10 to 'auto' in 1.4. Set the value of 'n_init' explicitly to suppress the warning
fashion-mnist-analysis | super()._check_params_vs_input(X, default_n_init=10)
fashion-mnist-analysis | ✓ Gráfico de subclusters guardado en carpetas: 5
fashion-mnist-analysis | ✓ Imágenes representativas guardadas en carpetas: 5
fashion-mnist-analysis | =====
fashion-mnist-analysis | Analizando subclusters para categoria 7
fashion-mnist-analysis | =====
fashion-mnist-analysis | Analizando cluster 7...
fashion-mnist-analysis | /usr/local/lib/python3.9/site-packages/sklearn/cluster/_kmeans.py:1412: FutureWarning: The default value of 'n_init' will change from 10 to 'auto' in 1.4. Set the value of 'n_init' explicitly to suppress the warning
fashion-mnist-analysis | super()._check_params_vs_input(X, default_n_init=10)
fashion-mnist-analysis | ✓ Gráfico de subclusters guardado en carpetas: 7
fashion-mnist-analysis | ✓ Imágenes representativas guardadas en carpetas: 7
fashion-mnist-analysis | =====
fashion-mnist-analysis | Analizando subclusters para categoria 0
fashion-mnist-analysis | =====
fashion-mnist-analysis | Analizando cluster 0...
fashion-mnist-analysis | /usr/local/lib/python3.9/site-packages/sklearn/cluster/_kmeans.py:1412: FutureWarning: The default value of 'n_init' will change from 10 to 'auto' in 1.4. Set the value of 'n_init' explicitly to suppress the warning
fashion-mnist-analysis | super()._check_params_vs_input(X, default_n_init=10)
fashion-mnist-analysis | ✓ Gráfico de subclusters guardado en carpetas: 0
fashion-mnist-analysis | ✓ Imágenes representativas guardadas en carpetas: 0
fashion-mnist-analysis | =====
fashion-mnist-analysis | # Análisis completado! Revisa los archivos en las carpetas 'results/' e 'images/'
fashion-mnist-analysis exited with code 0
```

Para la elección de un cluster en específico, elegi los 5, 7 y 0 (sandal, sneaker y t-shirt respectivamente)

Las imágenes se guardaron en las carpetas “results” y “images”

Actividad #3

Código main.py:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from umap import UMAP
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics.pairwise import euclidean_distances
import os

# Configuración de estilo
plt.style.use('default')
sns.set_palette("husl")

class FashionMNISTAnalyzer:
    def __init__(self):
        self.df = None
        self.X = None
        self.y = None
        self.umap_2d = None
        self.umap_results = None

    def load_data(self, csv_path):
        """Cargar datos desde archivo CSV"""
        print("Cargando datos...")
        self.df = pd.read_csv(csv_path)

        # Separar características y etiquetas
        self.y = self.df.iloc[:, 0].values # Primera columna son las
        etiquetas
        self.X = self.df.iloc[:, 1:].values # Resto son píxeles

        print(f"Datos cargados: {self.X.shape[0]} muestras, {self.X.shape[1]}
        características")
        print(f"Distribución de etiquetas: {np.unique(self.y,
        return_counts=True)}")

    def apply_umap_full(self, n_samples=5000):
        """Aplicar UMAP al conjunto completo (muestreado para eficiencia)"""
        print("Aplicando UMAP al conjunto completo...")

        # Muestrear para hacerlo manejable
        if len(self.X) > n_samples:
            indices = np.random.choice(len(self.X), n_samples, replace=False)
```

```

        X_sample = self.X[indices]
        y_sample = self.y[indices]
    else:
        X_sample = self.X
        y_sample = self.y

    # Estandarizar datos
    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X_sample)

    # Aplicar UMAP
    self.umap_2d = UMAP(n_components=2, random_state=42, n_neighbors=15,
min_dist=0.1)
    self.umap_results = self.umap_2d.fit_transform(X_scaled)

    self.visualize_full_dataset(self.umap_results, y_sample)
    return self.umap_results, y_sample

def visualize_full_dataset(self, umap_results, labels):
    """Visualizar resultados de UMAP para el conjunto completo"""
    plt.figure(figsize=(12, 8))

    # Diccionario de etiquetas Fashion-MNIST
    label_names = {
        0: 'T-shirt/top', 1: 'Trouser', 2: 'Pullover', 3: 'Dress',
        4: 'Coat', 5: 'Sandal', 6: 'Shirt', 7: 'Sneaker', 8: 'Bag', 9:
'Ankle boot'
    }

    scatter = plt.scatter(umap_results[:, 0], umap_results[:, 1],
                        c=labels, cmap='tab10', alpha=0.7, s=10)
    plt.colorbar(scatter, label='Categorías')
    plt.title('UMAP - Fashion-MNIST (Visualización 2D)')
    plt.xlabel('UMAP 1')
    plt.ylabel('UMAP 2')

    # Crear leyenda personalizada
    handles, _ = scatter.legend_elements()
    plt.legend(handles, [label_names[i] for i in range(10)],
                title="Categorías", bbox_to_anchor=(1.05, 1), loc='upper
left')

    plt.tight_layout()
    # GUARDAR EN CARPETAS

```

Actividad #3

```
plt.savefig('results/umap_full_dataset.png', dpi=300,
bbox_inches='tight')
plt.savefig('images/umap_full_dataset.png', dpi=300,
bbox_inches='tight')
print("☑ Gráfica guardada en: results/ e images/")
plt.show()

def analyze_cluster(self, cluster_label, n_samples=1000):
    """Analizar un cluster específico y encontrar subclusters"""
    print(f"Analizando cluster {cluster_label}...")

    # Filtrar instancias del cluster
    cluster_indices = np.where(self.y == cluster_label)[0]

    if len(cluster_indices) > n_samples:
        cluster_indices = np.random.choice(cluster_indices, n_samples,
replace=False)

    X_cluster = self.X[cluster_indices]

    # Aplicar UMAP al cluster
    scaler = StandardScaler()
    X_cluster_scaled = scaler.fit_transform(X_cluster)

    umap_cluster = UMAP(n_components=2, random_state=42, n_neighbors=10,
min_dist=0.1)
    cluster_results = umap_cluster.fit_transform(X_cluster_scaled)

    # Encontrar subclusters usando clustering
    kmeans = KMeans(n_clusters=3, random_state=42)
    subcluster_labels = kmeans.fit_predict(cluster_results)

    self.visualize_subclusters(cluster_results, subcluster_labels,
cluster_label, X_cluster)

    return cluster_results, subcluster_labels, X_cluster

def visualize_subclusters(self, umap_results, subcluster_labels,
cluster_label, X_data):
    """Visualizar subclusters dentro de un cluster principal"""
    label_names = {
        0: 'T-shirt/top', 1: 'Trouser', 2: 'Pullover', 3: 'Dress',
        4: 'Coat', 5: 'Sandal', 6: 'Shirt', 7: 'Sneaker', 8: 'Bag', 9:
'Ankle boot'
    }
```



```

# FIGURA 1: Gráfico de puntos de subclusters
plt.figure(figsize=(12, 8))
scatter = plt.scatter(umap_results[:, 0], umap_results[:, 1],
                      c=subcluster_labels, cmap='viridis', alpha=0.7,
s=40)
plt.title(f'Subclusters en {label_names[cluster_label]} (Categoría
{cluster_label})',
          fontsize=14, fontweight='bold')
plt.xlabel('UMAP Componente 1')
plt.ylabel('UMAP Componente 2')
plt.grid(True, alpha=0.3)
plt.colorbar(scatter, label='Subcluster')
plt.tight_layout()
# GUARDAR EN CARPETAS
plt.savefig(f'results/subclusters_category_{cluster_label}.png',
dpi=300, bbox_inches='tight')
plt.savefig(f'images/subclusters_category_{cluster_label}.png',
dpi=300, bbox_inches='tight')
print(f"☑ Gráfico de subclusters guardado en carpetas:
{cluster_label}")
plt.show()

# FIGURA 2: Imágenes representativas
self.display_representative_images(X_data, subcluster_labels,
cluster_label)

def display_representative_images(self, X_data, subcluster_labels,
cluster_label):
    """Mostrar imágenes representativas de cada subcluster"""
    unique_subclusters = np.unique(subcluster_labels)

    label_names = {
        0: 'T-shirt/top', 1: 'Trouser', 2: 'Pullover', 3: 'Dress',
        4: 'Coat', 5: 'Sandal', 6: 'Shirt', 7: 'Sneaker', 8: 'Bag', 9:
'Ankle boot'
    }

    images_per_row = 3
    total_images = len(unique_subclusters) * images_per_row

    fig, axes = plt.subplots(len(unique_subclusters), images_per_row,
                             figsize=(12, 4 * len(unique_subclusters)))

    if len(unique_subclusters) == 1:

```

```

        axes = [axes]

    for i, subcluster in enumerate(unique_subclusters):
        subcluster_indices = np.where(subcluster_labels == subcluster)[0]
        X_subcluster = X_data[subcluster_indices]

        # Calcular centroide
        centroid = np.mean(X_subcluster, axis=0)

        # Encontrar imágenes más cercanas al centroide
        distances = euclidean_distances(X_subcluster,
[centroid]).flatten()
        closest_indices = np.argsort(distances)[:images_per_row]

        for j, idx in enumerate(closest_indices):
            img = X_subcluster[idx].reshape(28, 28)
            axes[i][j].imshow(img, cmap='gray')
            axes[i][j].set_title(f'Subcluster {subcluster}\nImagen {j+1}')
            axes[i][j].axis('off')

    plt.suptitle(f'Imágenes representativas -
{label_names[cluster_label]}', fontsize=16)
    plt.tight_layout()
    # GUARDAR EN CARPETAS
    plt.savefig(f'results/representative_images_{cluster_label}.png',
dpi=300, bbox_inches='tight')
    plt.savefig(f'images/representative_images_{cluster_label}.png',
dpi=300, bbox_inches='tight')
    print(f"☑ Imágenes representativas guardadas en carpetas:
{cluster_label}")
    plt.show()

def main():
    # Crear carpetas si no existen
    os.makedirs('results', exist_ok=True)
    os.makedirs('images', exist_ok=True)

    analyzer = FashionMNISTAnalyzer()

    # Cargar datos
    csv_path = 'data/fashion-mnist.csv'

    if not os.path.exists(csv_path):
        print(f"Error: No se encuentra el archivo {csv_path}")

```

```

        print("Por favor, asegúrate de que el archivo CSV esté en la carpeta
data/")
        return

    analyzer.load_data(csv_path)

    # Análisis del conjunto completo
    umap_results, labels = analyzer.apply_umap_full(n_samples=5000)

    # Analizar MÚLTIPLES clusters específicos
    clusters_to_analyze = [5, 7, 0] # Sandals, Sneakers, T-shirts

    for cluster_id in clusters_to_analyze:
        print(f"\n" + "="*50)
        print(f"Analizando subclusters para categoría {cluster_id}")
        print("="*50)
        cluster_results, subcluster_labels, X_cluster =
analyzer.analyze_cluster(cluster_id)

        print("\n🐼 Análisis completado! Revisa los archivos en las carpetas
'results/' e 'images/'")

if __name__ == "__main__":
    main()

```

.dockerignore

.git

__pycache__

*.pyc

.DS_Store

.env

README.md

*.log

*.tmp

.ipynb_checkpoints

.vscode

.idea

Actividad #3

docker-compose.yml:

```
version: '3.8'

services:
  fashion-analysis:
    build: .
    volumes:
      - ./app
      - ./data:/app/data
      - ./results:/app/results
      - ./images:/app/images
    working_dir: /app
    stdin_open: true
    tty: true
    container_name: fashion-mnist-analysis
```

requirements.txt:

```
pandas==2.0.3
numpy==1.24.3
matplotlib==3.7.1
seaborn==0.12.2
scikit-learn==1.3.0
umap-learn==0.5.3
plotly==5.14.1
jupyter==1.0.0
notebook==6.5.4
opencv-python==4.8.0.74
Pillow==9.5.0
tqdm==4.65.0
```

Dockerfile:

```
# Dockerfile para Análisis de Fashion-MNIST con UMAP - VERSIÓN CORREGIDA
FROM python:3.9-slim

# Metadatos
LABEL maintainer="Ale"
LABEL description="Análisis de clustering con UMAP en Fashion-MNIST"
LABEL version="1.0"

# Variables de entorno
ENV PYTHONUNBUFFERED=1
ENV PYTHONPATH=/app

# Directorio de trabajo
WORKDIR /app
```

```

# Actualizar lista de paquetes PRIMERO y manejar errores gracefully
RUN apt-get update || true && \
  apt-get install -y --no-install-recommends \
  build-essential \
  curl \
  && apt-get clean \
  && rm -rf /var/lib/apt/lists/*

# Copiar e instalar dependencias de Python
COPY requirements.txt .
RUN pip install --no-cache-dir --upgrade pip && \
  pip install --no-cache-dir -r requirements.txt

# Copiar el código de la aplicación
COPY . .

# Crear directorios necesarios
RUN mkdir -p data results images

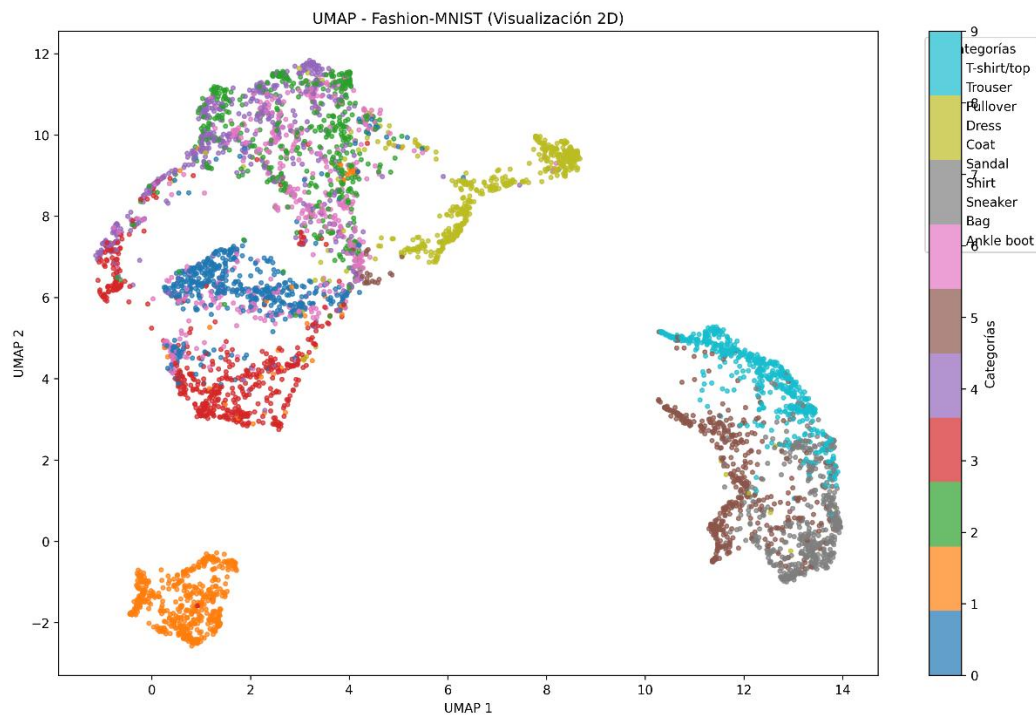
# Comando por defecto
CMD ["python", "main.py"]

```

Extra:

Se utilizo la base de datos Fashion-mnist test, se renombro a fashion-mnist.cv

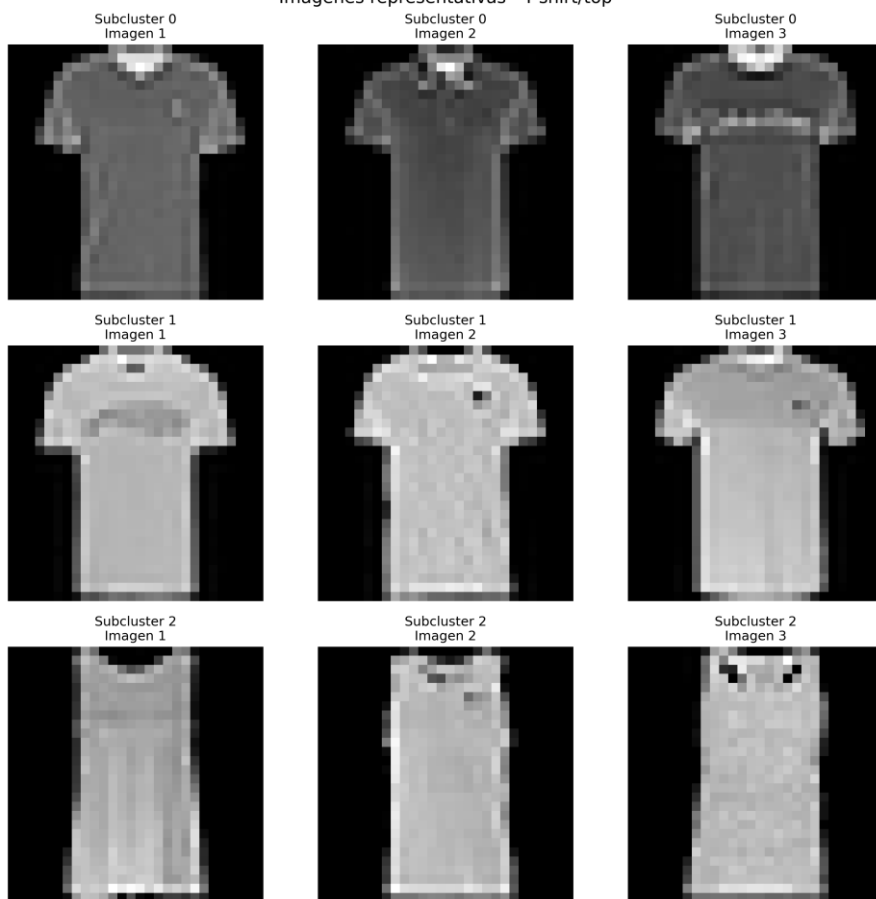
Imágenes de resultados:



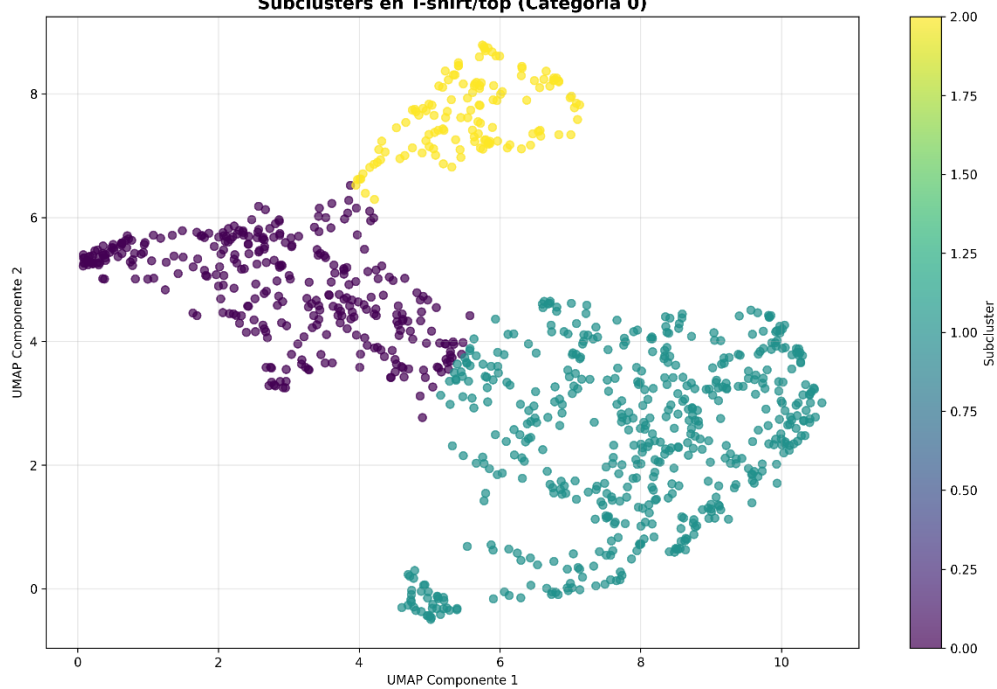
Actividad #3

T-shirt:

Imágenes representativas - T-shirt/top

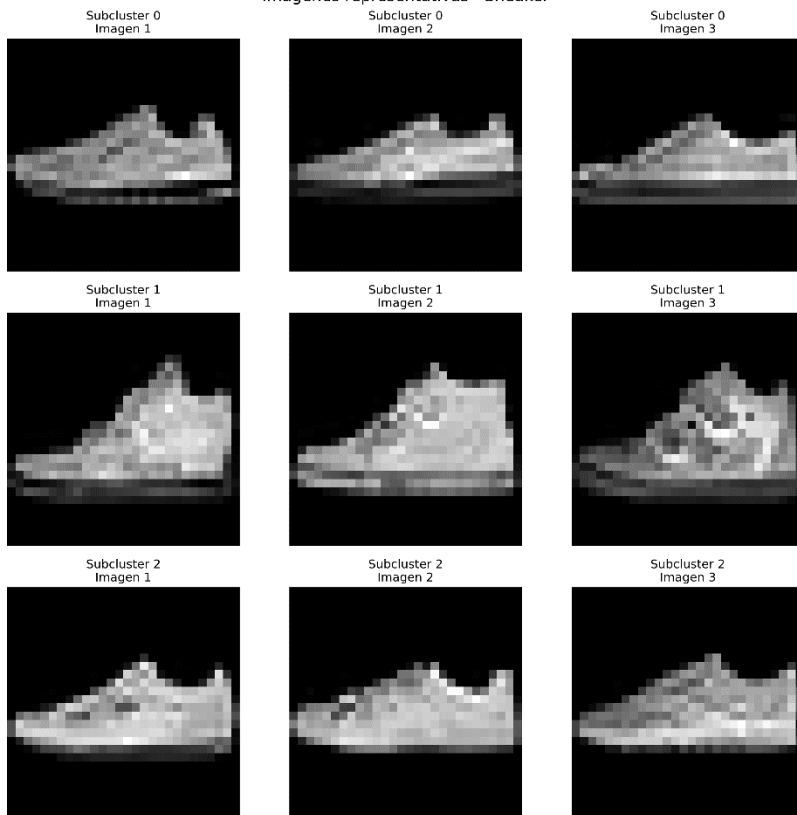


Subclusters en T-shirt/top (Categoría 0)

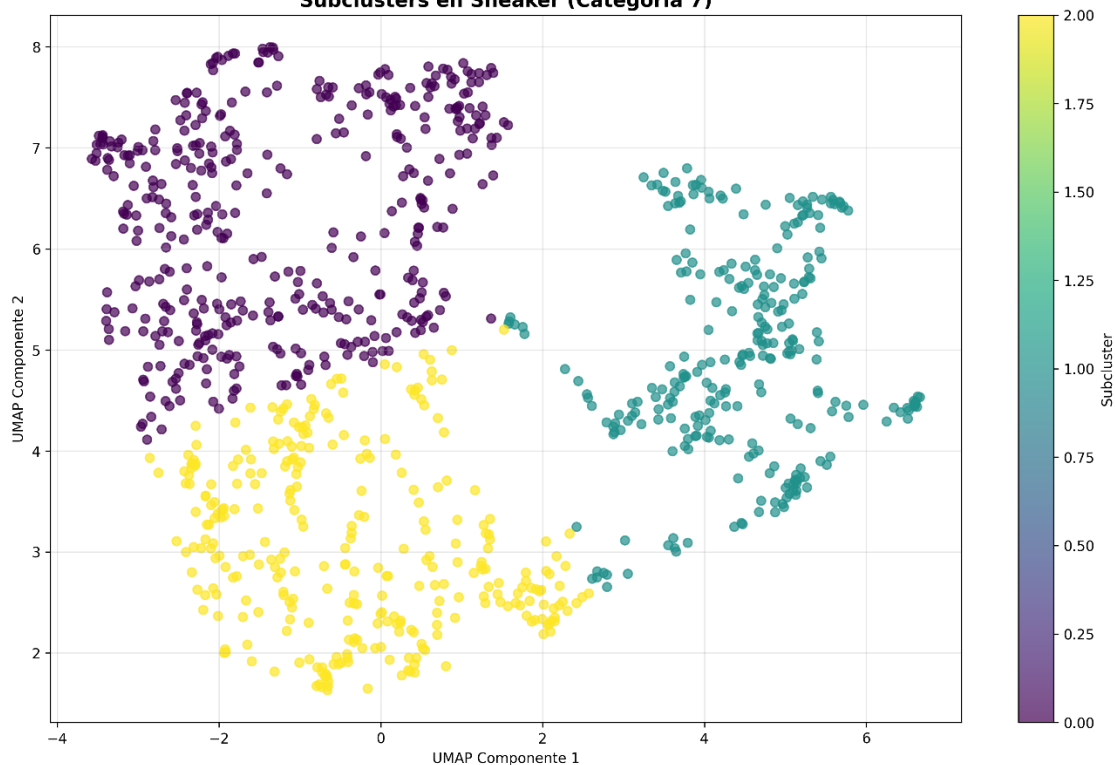


Sneakers:

Imágenes representativas - Sneaker

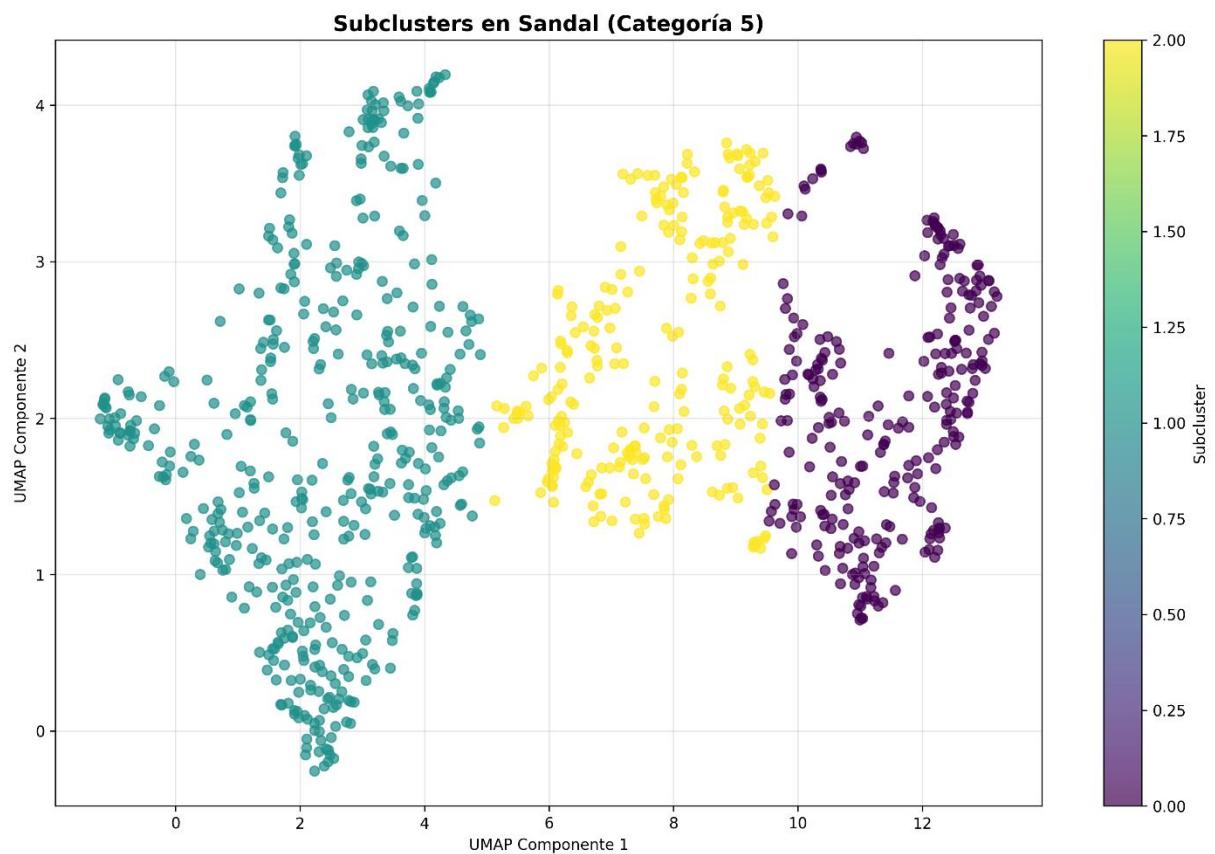
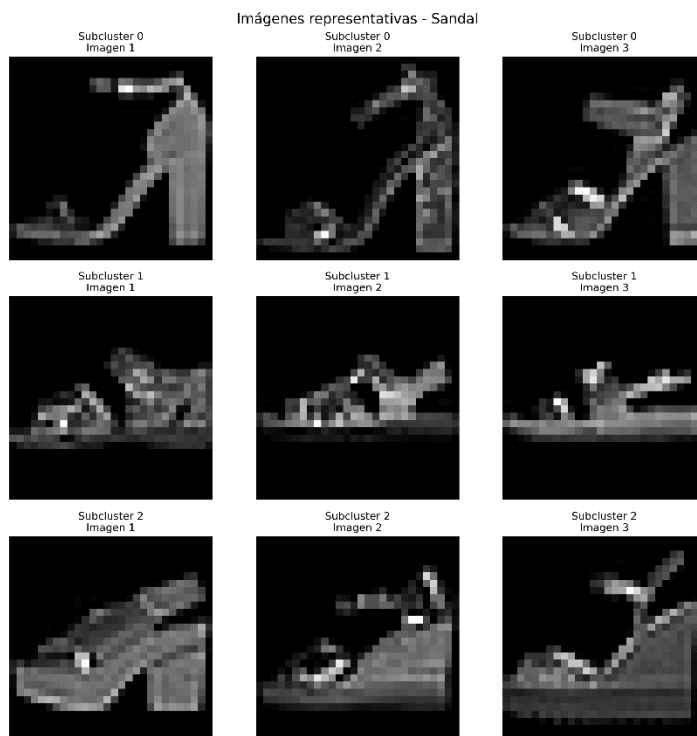


Subclusters en Sneaker (Categoría 7)



Actividad #3

Sandels:



Conclusión

Realmente, a mi punto de vista se me hace interesante la dispersión de los datos, pero como cada subclusters tienen puntos claves en donde se concentran mas datos, y puntos concretos donde son mas lejanos, los que mas me interesan en lo personal son los que están muy cerca como casi conectados (imaginando vaya que cada cluster/subcluster hace referencia a cada prenda identificada por medio de la base de datos, documentación y extracción de imágenes), tiene bastante limitantes como la generación de imágenes 2d de 28x28 pixeles solo a escalas de blanco y negro, se generaliza bastante los datos, realmente no podría diferenciar cuales son los elementos específicos de cada sección, haría falta etiquetas,

Actividad #3

Referencias

Fashion MNIST. (2017, December 7). <https://www.kaggle.com/datasets/zalando-research/fashionmnist>

Probst, D. (n.d.). *tmap - Visualize big high-dimensional data*. <https://tmap.gdb.tools/>

Murel, J., PhD, & Kavlakoglu, E. (2025, 18 febrero). Reducción de la dimensionalidad. ¿Qué es la reducción de la dimensionalidad? <https://www.ibm.com/mx-es/think/topics/dimensionality-reduction>

pandas documentation — pandas 2.3.3 documentation. (n.d.). <https://pandas.pydata.org/docs/>

NumPy documentation — NumPy v2.3 Manual. (n.d.). <https://numpy.org/doc/stable/>

Matplotlib — Visualization with Python. (n.d.). <https://matplotlib.org/>

seaborn: statistical data visualization — seaborn 0.13.2 documentation. (n.d.). <https://seaborn.pydata.org/>

UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction — umap 0.5.8 documentation. (n.d.). <https://umap-learn.readthedocs.io/en/latest/>

scikit-learn: machine learning in Python — scikit-learn 1.7.2 documentation. (n.d.). <https://scikit-learn.org/stable/>

os — Interfaces misceláneas del sistema operativo — documentación de Python - 3.10.18. (n.d.). <https://docs.python.org/es/3.10/library/os.html>